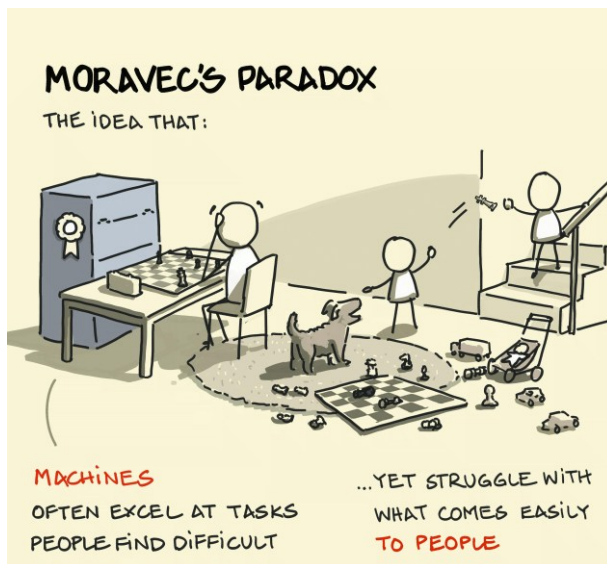


Vision Language Action Models for Safety

• Moravec's Paradox

- What is hard for humans is easy for AI, and what is easy for humans is hard for AI.
e.g. reasoning, logic, chess (**easy**) $\leftrightarrow$ walking, grasping, avoiding (**hard**)



- **Why so hard to pick up and replace a cup for robot?**

- Sensorimotor skill is a deep layer optimized over a billion years
- **The more effortless something looks, the more deeply it was engineered and the harder it is to reverse-engineer. E.g (Avoid Obstacle is easy for human but hard for robot!)**

Contact & force

soft cups, unknown friction -> apply just the right force

Dexterity

in-hand manipulation, re-grasping with multi-finger hands

Deformable & fluids

fold cloth, tie rope, pour water -> modeling breaks

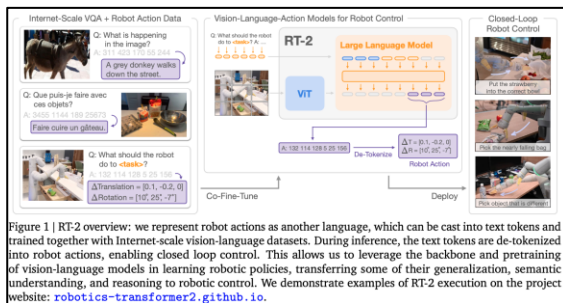
Long horizon

one cup is fine; "tidy the kitchen" is dozens of steps, no failures

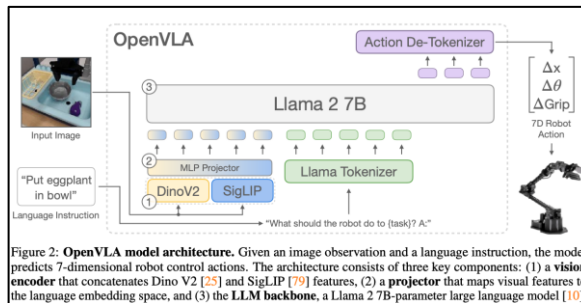
Vision Language Action models for Safety

- But, VLA already know what to do
 - Perception and Semantics are no longer the bottleneck
 - VLA models can follow language instructions in the place where has never seen.

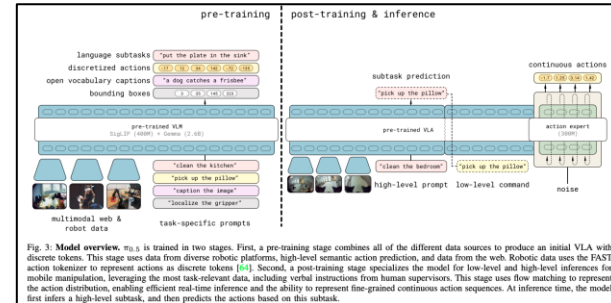
RT-2



OpenVLA



Pi 0.5



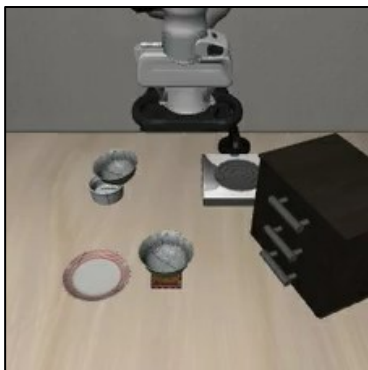
Web knowledge
→ semantics transfer to action

pretrained on cross-embodiment
robot data
→ adaptable to new robots

Continuous action, flow matching,
long-horizon manipulation
→ works in an unseen home

• Then Why VLA actions still hits obstacles?

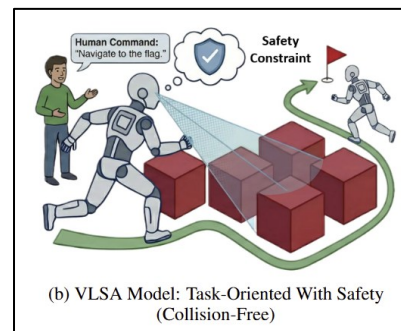
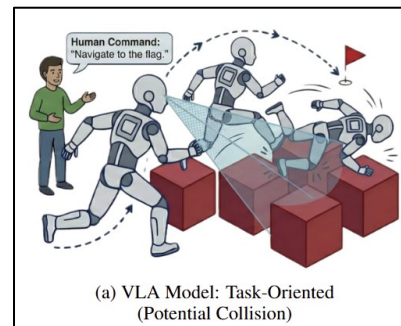
- Demonstrations are already collision-free
- The loss does not distinguish a violation
- **Avoidance is a standing condition, not a task**



Without obstacle



With obstacle



- **So how do we make it avoid obstacles?**

A. Train it in policy

SafeVLA · constrained RL · preference optimization

- add a penalty or constraint to the loss
- avoidance becomes one more downstream task
- new obstacle or new scene → collect and retrain
- a 7B fine-tune for what geometry already answers

✗ **guarantee stays soft — a penalty, not enforced**

✓ **can reach hazards geometry cannot express**

B. Filter it out (Post-hoc)

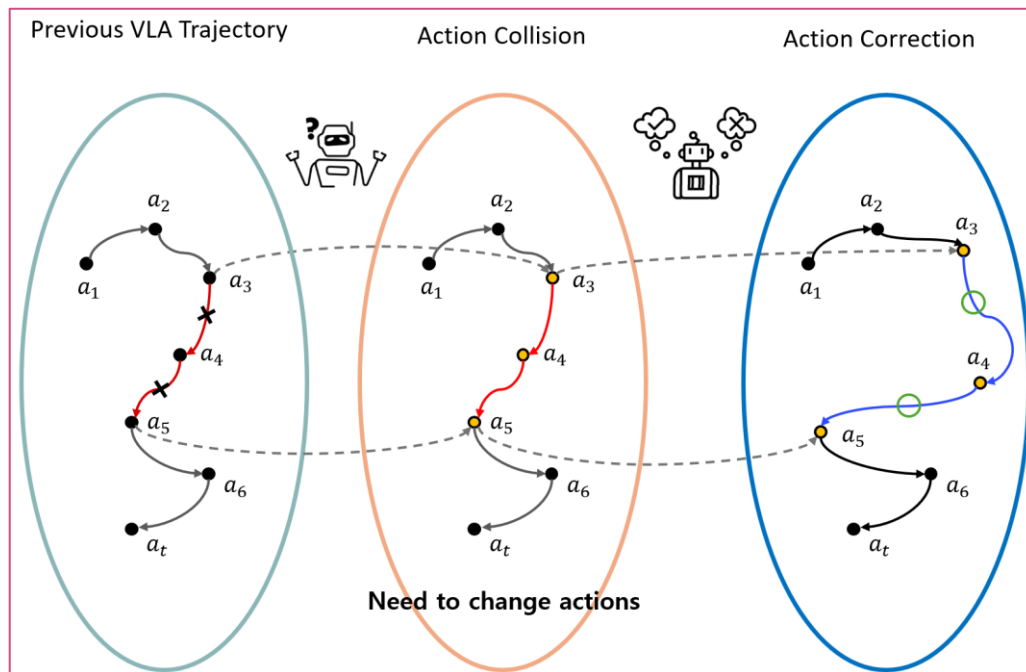
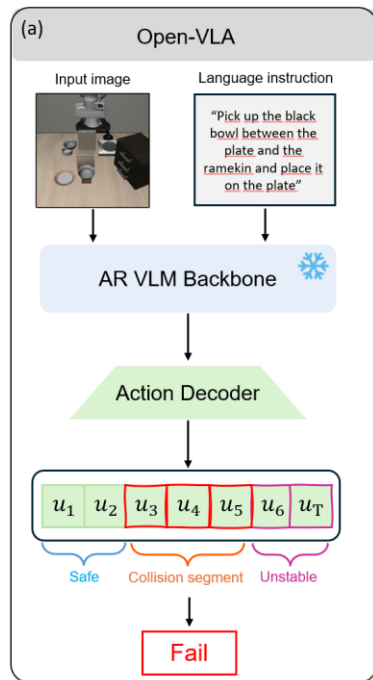
CBF-QP safety filter

- repair the action after the policy, before execution
- policy weights untouched, plug-and-play
- passes the action through when it is already safe
- one scene-level geometry, no task-specific data

✓ **guarantee is hard — forward invariance**

✗ **only as good as the perception feeding it**

- So how do we make it avoid obstacles?



Vision Language Action models for Safety



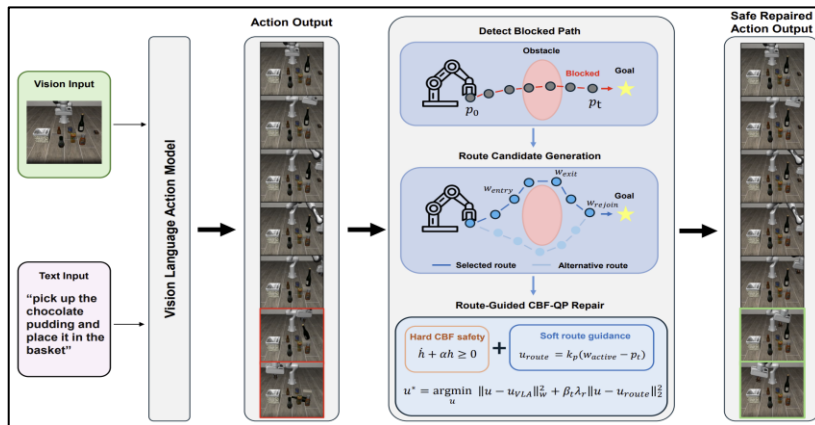
LG Electronics
Toronto AI Lab

Route-Guided CBF-QP Repair for Safe Vision-Language-Action Models

Chan Park^{1,3}, Sunwoo Hong^{2,3}, Kirak Kim^{2,3}, Yoonjeong Park^{2,3},
Alexander W. Olson³, Ali Pesaranhader^{4,*}

¹Sungkyunkwan University · ²KAIST · ³University of Toronto · ⁴LG Electronics, Toronto AI Lab
* Corresponding author

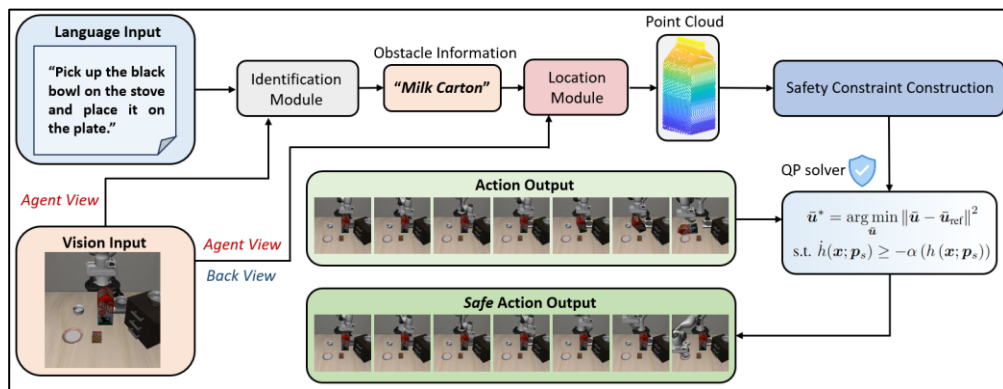
RSS 2026 Workshop on Trustworthy Embodied Foundation Models



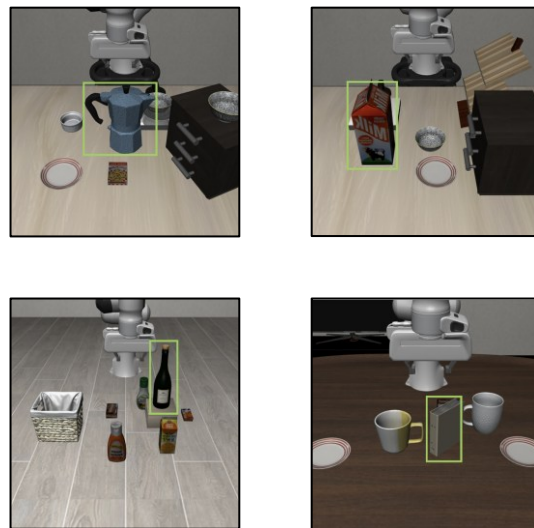
Overview

Related work

AEGIS model

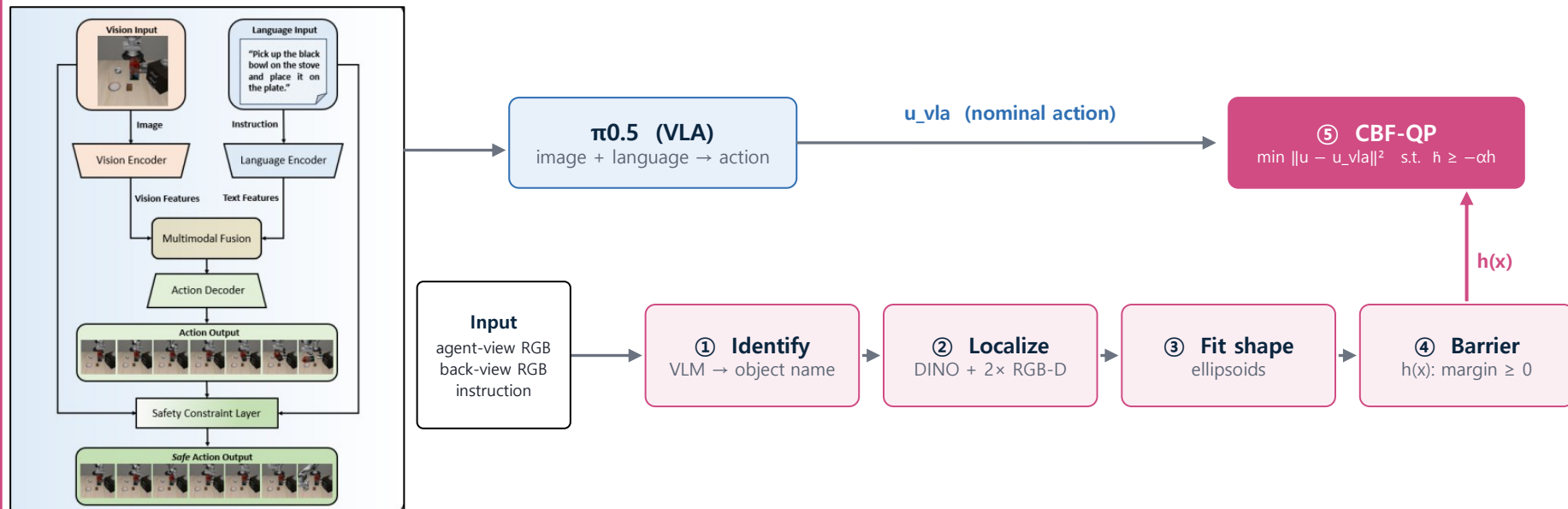


SafeLIBERO



obstacles: moka pot, book, milk carton, wine bottle

Vision Language Action models for Safety



Vision Language Action models for Safety

① Identify

② Localize

③ Fit shape

④ Barrier $h(x)$

⑤ Repair action

in front-view RGB + instruction



out exactly one object name (type + color)



"Pick up the black bowl on the stove and place it on the plate."

Prompt to the VLM (GLM-4.5V, GPT, Gemini)

The robot must follow this instruction: [Instruction].
Based on both the instruction and the image, identify exactly one non-robot object that is most likely to obstruct the robot's motion during task execution. Output a uniquely identifiable obstacle name including both color and object type ...

Output only the object name.



"Milk Carton"

Vision Language Action models for Safety

① Identify

② Localize

③ Fit shape

④ Barrier $h(x)$

⑤ Repair action

in object name + RGB-D from agent view and back view → **out** point cloud in the world frame



front view



back view

GroundingDINO: "Milk Carton" as text query

pixels in the box → 3D points Cloud

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = T_{\text{cam}}^{\text{world}} \cdot \begin{bmatrix} K^{-1} \cdot d \cdot \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \\ 1 \end{bmatrix}$$

(u, v): pixel

d: depth from the sensor

K^{-1} pixel → ray in the camera frame

T camera → world, so both views land in one frame



Vision Language Action models for Safety

① Identify

② Localize

③ Fit shape

④ Barrier $h(x)$

⑤ Repair action

in point cloud (obstacle); fixed size (end-effector)



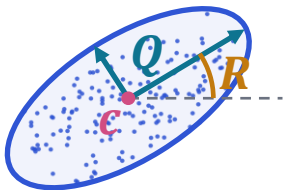
out ellipsoid $E = \{ c, Q, R \}$

what an ellipsoid is

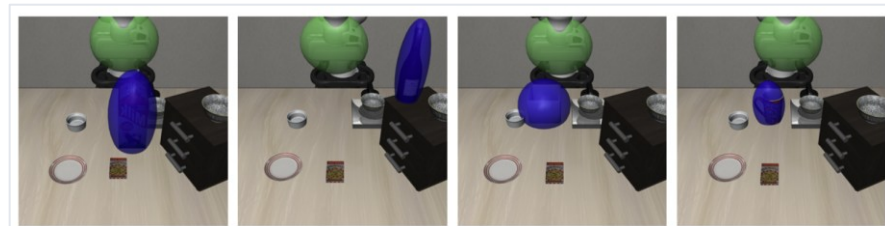
$$\mathcal{E} = \{ x : (x - c)^T R^T Q R (x - c) \leq 1 \}$$

c center
 R orientation (tilt)
 Q axis lengths

how it is fitted



$$\begin{aligned} \min_{c, R \in SO(d)} & -\log \det(Q) \\ \text{subject to} & \begin{bmatrix} Q & R(x_i - c) \\ (R(x_i - c))^T & 1 \end{bmatrix} \geq 0, \quad i = 1, 2, \dots, n \\ & R^T R = I, \quad \det(R) = 1, Q > 0. \end{aligned}$$



blue: obstacle ellipsoid

green: end-effector ellipsoid

end-effector

(0.06, 0.12, 0.11) m, fixed to the gripper pose

- Obstacle and end-effector both become ellipsoids

Vision Language Action models for Safety

① Identify

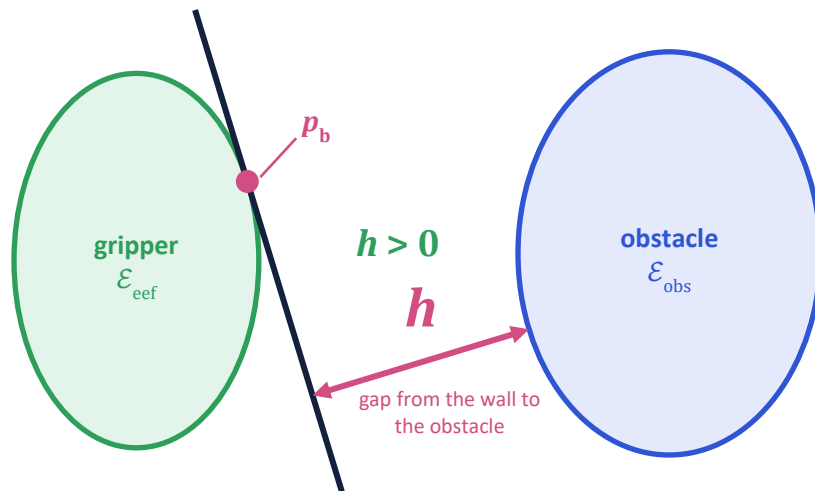
② Localize

③ Fit shape

④ Barrier $h(x)$

⑤ Repair action

Why To constrain motion, we need one differentiable number that says how safe the robot is right now.



$$h > 0$$

safe: there is a gap



$$h = 0$$

on the boundary



$$h < 0$$

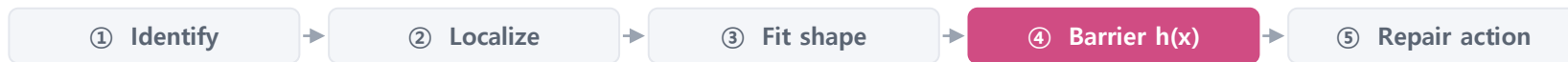
collision

$$\mathcal{C} = \{ x : h(x) \geq 0 \}$$

the safe set

→ Goal: keep $h(x) \geq 0$

Vision Language Action models for Safety



Why Stopping at $h = 0$ is too late. We command velocity, not position, so we limit how fast the gap may close.

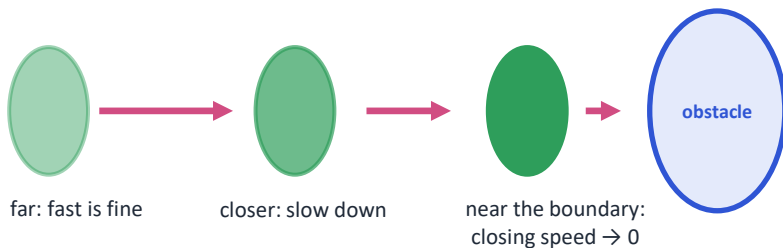
$$\dot{h}(x, u) \geq -\alpha \cdot h(x)$$

how fast the gap changes
(negative = closing in)

how strict

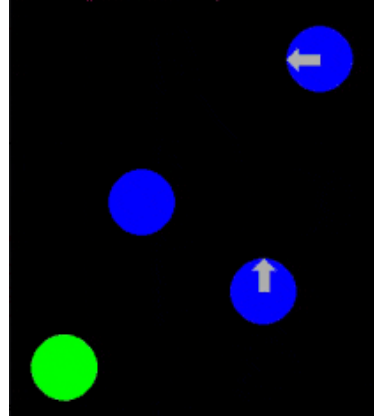
the gap that is left

closing speed $\leq \alpha \times$ remaining gap

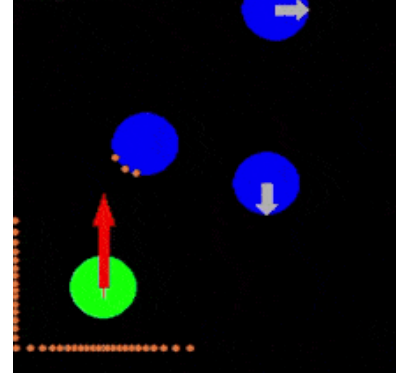


Control barrier function demo

CBF OFF (press x to turn on)



CBF ON (press x to turn off)
CBF alpha: 0.1 (press z to change)
Fixed dir: False, CBF patrols: False (c/v)
Simulate lidar: True



Vision Language Action models for Safety

① Identify

② Localize

③ Fit shape

④ Barrier $h(x)$

⑤ CBF-QP Repair action

QP(Quadratic Programming): optimization problem with a quadratic objective and linear constraints

$$u_{\text{safe}} = \arg \min_u$$

$$\|u - u_{\text{vla}}\|^2$$

s.t.

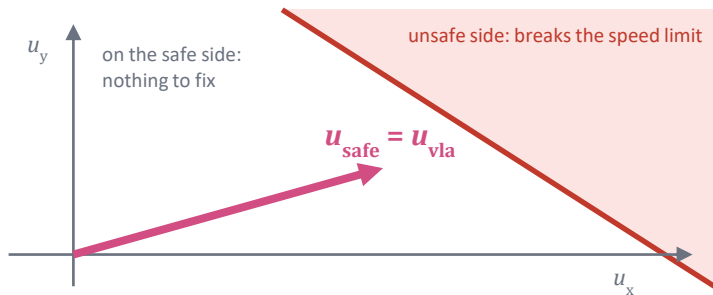
$$\dot{h}(x, u) \geq -\alpha h(x)$$

Objective: change the VLA's action as little as possible, so the task intent survives.

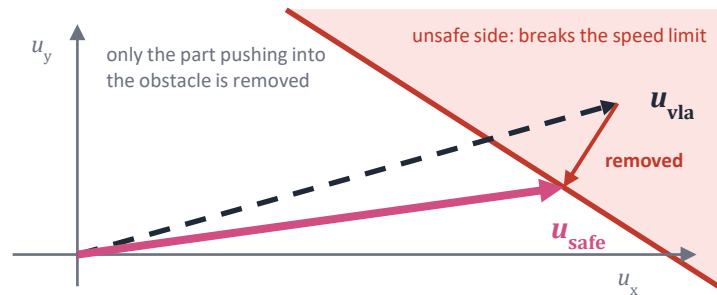
Constraint: the speed limit, never violated (hard). Linear in u , so it is one straight line.

0.36ms

Already safe → untouched

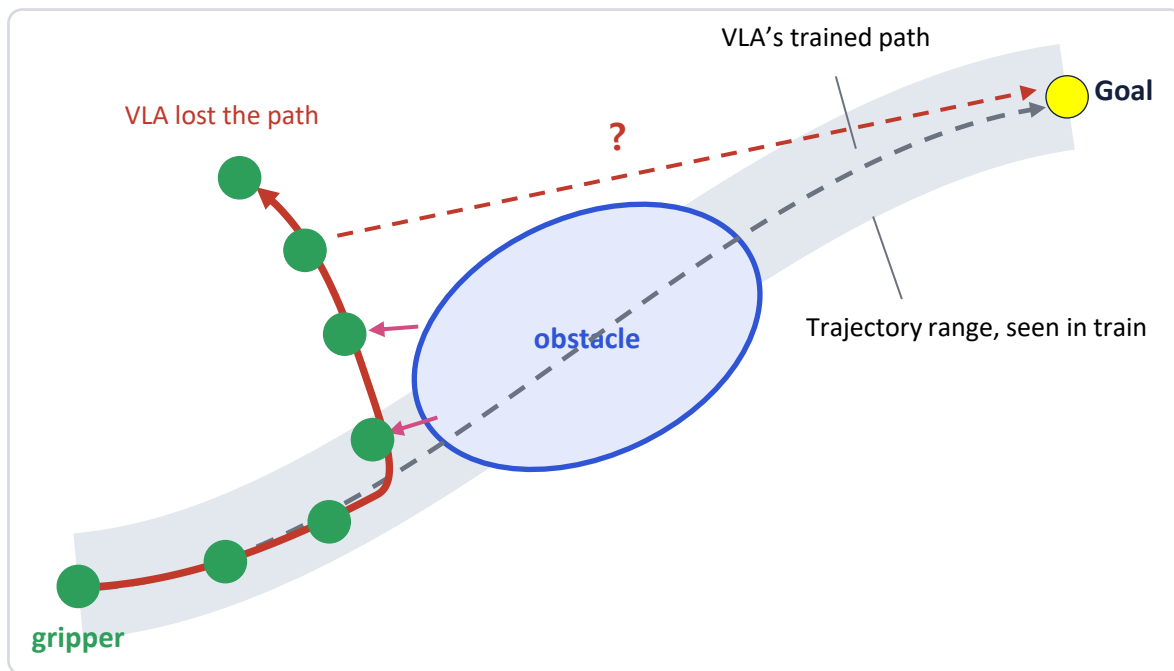


Unsafe → minimal repair



Limitation of AEGIS

Once out of distribution (OOD), the VLA fails to recover and return to the nominal trajectory

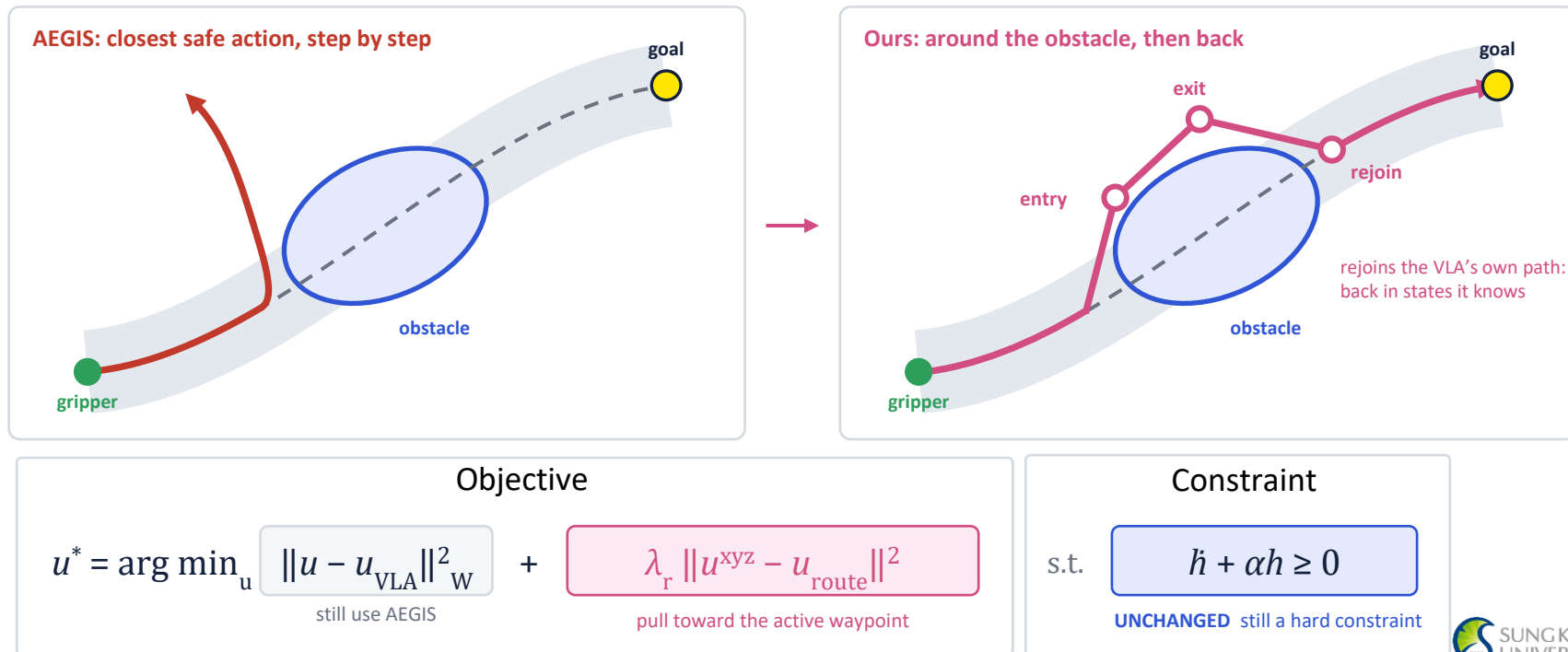


Safe action $\neq$ Successful task execution

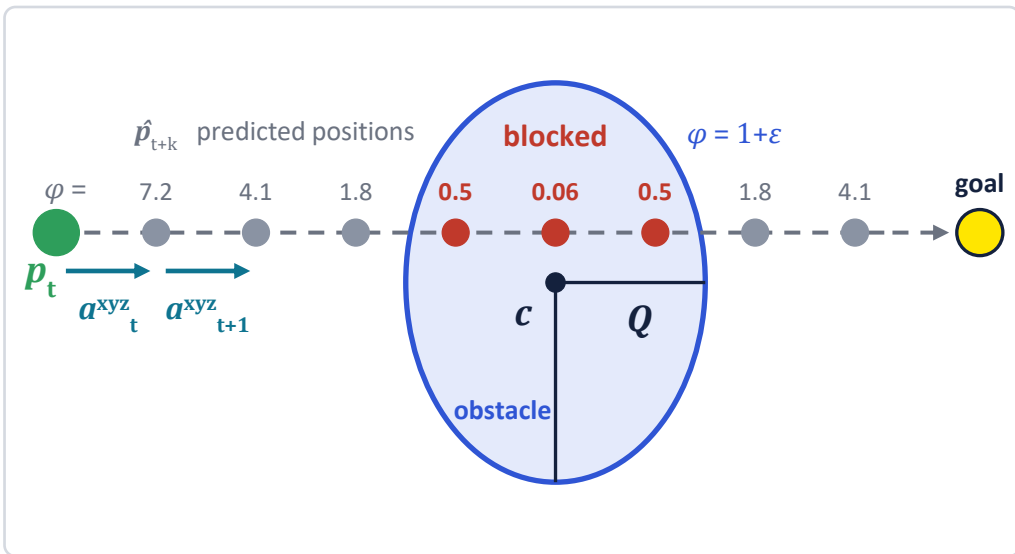
Pushed off the demonstrated states,
the VLA fails the task.

-> Safety-induced distribution shift

Method: Keep the CBF constraint, change the objective



Step 1: Is the path blocked?



Eq. 2 Where does the VLA intend to go?

$$\hat{p}_{t+k} = p_t + \sum_{i < k} a^{xyz}_{t+i}$$

Add up VLA's action chunk plan for the next H steps.

Eq. 3 How close is a point to the obstacle?

$$\phi(p) = (p - c)^T Q^{-1} (p - c)$$

Eq. 4 Blocked?

$$\min_k \phi(\hat{p}_{t+k}) < 1 + \epsilon \quad \epsilon = 0.05$$

e. g: $\phi = 0.50, 0.06, 0.50$ at $k = 4, 5, 6 \rightarrow$ blocked.

AEGIS reacts at the obstacle just before touch. The chunk lets us see the block H steps in advance.

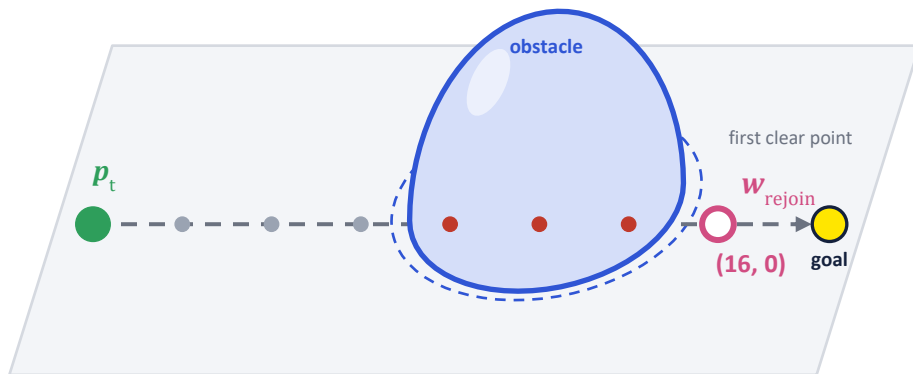
Step 2 · Where do the three waypoints go?

1 blocked?

2 waypoints

3 which side

4 follow



w_{rejoin} back onto the VLA's own path

1. first predict clear point after blocked point

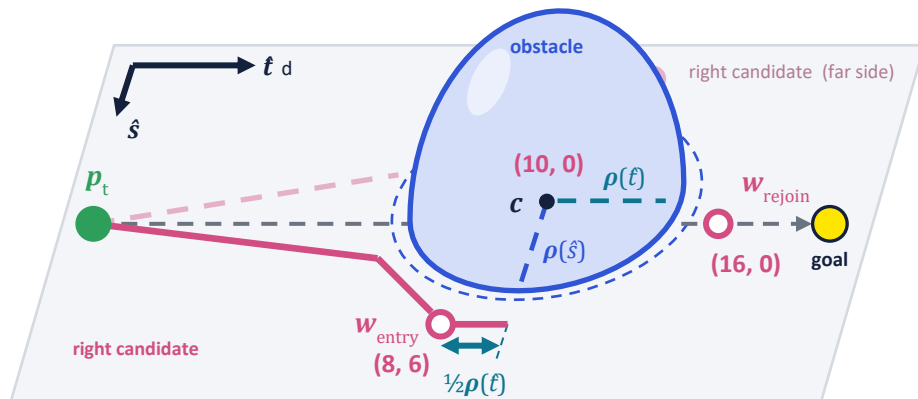
Step 2 · Where do the three waypoints go?

1 blocked?

2 waypoints

3 which side

4 follow



w_{rejoin} back onto the VLA's own path

1. first predict clear point after blocked point

w_{entry}, w_{exit} waypoint beside the obstacle $\rho(t) = 4$ $\rho(s) = 6$

$$w_{entry} = c - \frac{1}{2}\rho(t) + \rho(s)$$

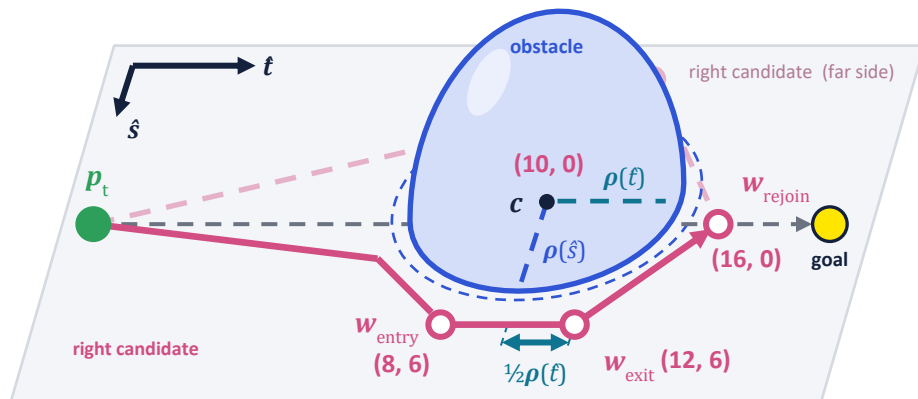
Step 2 · Where do the three waypoints go?

1 blocked?

2 waypoints

3 which side

4 follow



w_{rejoin} back onto the VLA's own path

1. first predict clear point after blocked point

w_{entry} , w_{exit} waypoint beside the obstacle $\rho(t) = 4$ $\rho(s) = 6$

$$w_{entry} = c - \frac{1}{2}\rho(t) + \rho(s)$$

$$w_{exit} = c + \frac{1}{2}\rho(t) + \rho(s)$$

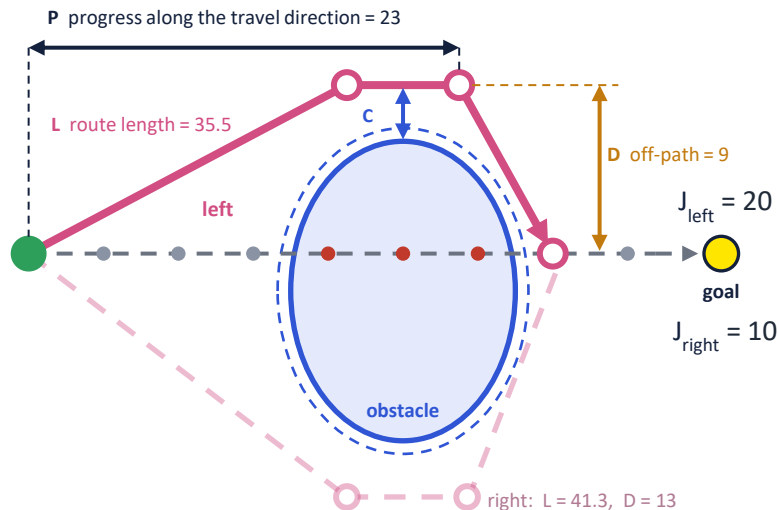
Step 3 · Which side to go?

1 blocked?

2 waypoints

3 which side

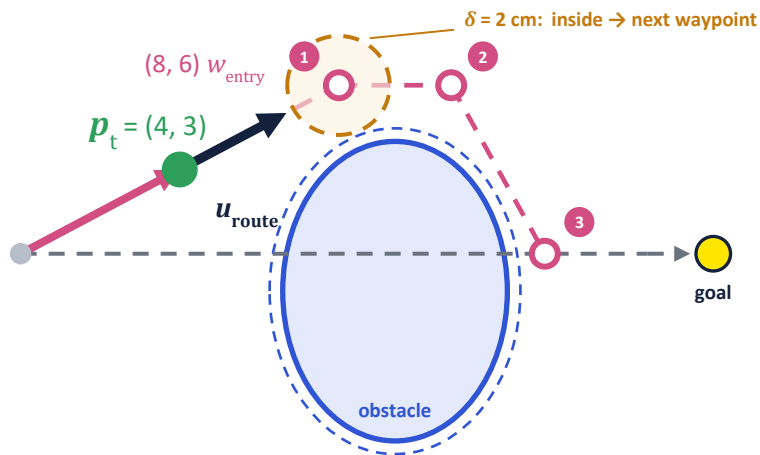
4 follow



$$\{left, right\} = \arg \max_s J(left, right), \quad J = C + P - L - D$$

term	meaning
C clearance	stays clear of the surface
P progress	still moves forward
L length	is short: fewer extra steps
D deviation	stays near the path the VLA knows
J	Score

Step 4 · Follow it: one waypoint at a time



1 blocked?

2 waypoints

3 which side

4 follow

① The pull a number given to the QP

$$u_{\text{route}} = (w_{\text{active}} - p_t) = (8-4, 6-3) \text{ cm} = (0.04, 0.03) \text{ m}$$

② The switch one waypoint at a time

$$\|p_t - w_{\text{active}}\| < \delta: \|(4, 3)\| = 5 > \delta$$

entry → exit → rejoin → off

③ Route-guided CBF-QP the final problem, solved every step

$$u^* = \arg \min_u \|u - u_{\text{VLA}}\|_W^2 + \lambda_r \|u^{\text{xyz}} - (w_{\text{active}} - p_t)\|^2$$

$$\text{s.t. } h(x, u) + \alpha h(x) \geq 0$$

Result: same episode, safety changes the outcome

AEGIS



Task Success + Collision

Ours (Route-Guided CBF-QP)



Task Success + Safe

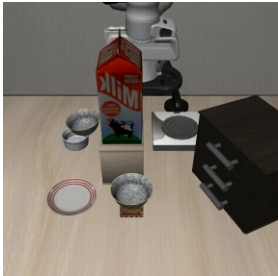
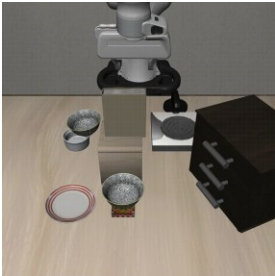
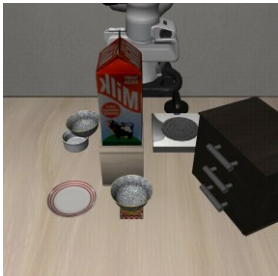
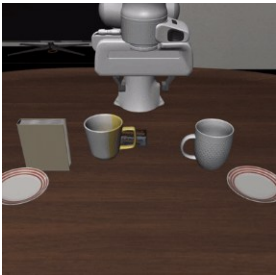
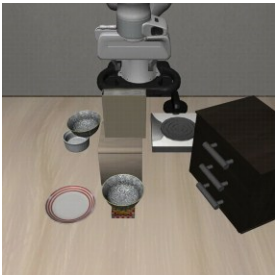
Experiment

TABLE I: Quantitative results on the SafeLIBERO benchmark.

Method	Metric	SafeLIBERO Suite				Average
		Spatial	Goal	Object	Long	
OpenVLA-OFT	CAR $\uparrow$ (%)	11.75	24.25	15.25	5.25	14.12
	TSR $\uparrow$ (%)	31.00	23.75	25.00	3.50	20.81
	SafeSuccess $\uparrow$ (%)	10.75	12.50	11.25	3.50	9.50
	ETS $\downarrow$	127.27	148.68	134.85	312.86	180.92
$\pi_{0.5}$	CAR $\uparrow$ (%)	14.25	23.75	23.00	8.00	17.25
	TSR $\uparrow$ (%)	38.25	42.50	33.75	12.50	31.75
	SafeSuccess $\uparrow$ (%)	12.50	18.00	19.25	5.75	13.88
	ETS $\downarrow$	162.61	138.31	112.01	422.21	208.79
$\pi_{0.5}$ + AEGIS	CAR $\uparrow$ (%)	63.75	83.75	56.25	55.00	64.69
	TSR $\uparrow$ (%)	60.00	85.00	67.50	38.75	62.81
	SafeSuccess $\uparrow$ (%)	52.50	73.75	41.25	21.25	47.19
	ETS $\downarrow$	85.61	104.33	67.68	181.85	109.87
$\pi_{0.5}$ + Route-Guided CBF-QP (Ours)	CAR $\uparrow$ (%)	66.25	81.25	76.25	52.50	69.06
	TSR $\uparrow$ (%)	65.00	80.00	72.50	42.50	65.00
	SafeSuccess $\uparrow$ (%)	53.75	67.50	55.00	22.50	49.69
	ETS $\downarrow$	91.20	129.50	65.10	188.00	118.45

Notes: CAR: Collision Avoidance Rate; TSR: Task Success Rate; SafeSuccess: successful task completion without collision; ETS: Execution Time Steps. Arrows indicate direction of improvement ($\uparrow$ higher is better, $\downarrow$ lower is better). Best results are highlighted in **bold**.

Vision Language Action models for Safety

	Goal Obstacle: Milk	Long Obstacle: yellow book	Spatial Obstacle: yellow book	Object Obstacle: wine bottle
AEGIS				
	Task Success + Collision	Task Fail + Collision	Task Fail + Collision	Task Success + Collision
Ours				
	Task Success + Safe	Task Success + Safe	Task Success + Safe	Task Success + Safe

• Limitation

- Building the obstacle model requires calibrated RGB-D plus a costly VLM/detector, so latency is dominated by perception.
- Waypoints are frozen after gating, so a poor rejoin is not corrected.
- Safety is enforced only for an EEF-centric ellipsoid, collisions involving arm links are not covered by the CBF